

Практическая часть

Установка зависимостей

1. Создание и активация виртуального окружения, установка компонентов

```

PROBLEMS  TERMINAL  OUTPUT  PORTS  DEBUG CONSOLE
PS C:\Users\Admin\Documents\fastapi_books> python -m venv venv
PS C:\Users\Admin\Documents\fastapi_books> venv\Scripts\activate
(venv) PS C:\Users\Admin\Documents\fastapi_books> pip install fastapi uvicorn[standard] pydantic
Collecting fastapi
  Downloading fastapi-0.120.0-py3-none-any.whl.metadata (28 kB)

```

Задание 1: Создание простого REST API

2. Создание простого REST API

```

main.py  X
main.py > get_books
1 from fastapi import FastAPI, HTTPException, status
2 from pydantic import BaseModel, Field
3 from typing import List, Optional
4 from datetime import datetime
5
6 app = FastAPI(
7     title="Books API",
8     description="REST API for managing a book library",
9     version="1.0.0"
10 )
11
12 class Book(BaseModel):
13     id: Optional[int] = None
14     title: str = Field(..., min_length=1, max_length=200, description="Book title")
15     author: str = Field(..., min_length=1, max_length=100, description="Book author")
16     year: int = Field(..., ge=1000, le=datetime.now().year, description="Publication year")
17     isbn: Optional[str] = Field(None, min_length=10, max_length=13, description="Book ISBN")
18
19     class Config:
20         json_schema_extra = {
21             "example": {
22                 "title": "The Master and Margarita",
23                 "author": "Mikhail Bulgakov",
24                 "year": 1967,
25                 "isbn": "9785170123456"
26             }
27         }
28
29 class BookUpdate(BaseModel):
30     title: Optional[str] = Field(None, min_length=1, max_length=200)
31     author: Optional[str] = Field(None, min_length=1, max_length=100)
32     year: Optional[int] = Field(None, ge=1000, le=datetime.now().year)
33     isbn: Optional[str] = Field(None, min_length=10, max_length=13)
34
35 books_db: List[Book] = [
36     Book(id=1, title="War and Peace", author="Leo Tolstoy", year=1869, isbn="9785170987654"),
37     Book(id=2, title="Crime and Punishment", author="Fyodor Dostoevsky", year=1866, isbn="9785170876543"),
38     Book(id=3, title="Eugene Onegin", author="Alexander Pushkin", year=1833, isbn="9785170765432")
39 ]
40 next_id = 4
41
42 @app.get("/", tags=["Root"])
43 async def root():
44     return {"message": "Welcome to Books API!", "docs": "/docs", "redoc": "/redoc"}
45

```

```

main.py X
main.py > ...
46 @app.get("/api/books", response_model=List[Book], tags=["Books"])
47 async def get_books():
48     return books_db
49
50 @app.get("/api/books/{book_id}", response_model=Book, tags=["Books"])
51 async def get_book(book_id: int):
52     for book in books_db:
53         if book.id == book_id:
54             return book
55     raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"Book with ID {book_id} not found")
56
57 @app.post("/api/books", response_model=Book, status_code=status.HTTP_201_CREATED, tags=["Books"])
58 async def create_book(book: Book):
59     global next_id
60     book.id = next_id
61     next_id += 1
62     books_db.append(book)
63     return book
64
65 @app.put("/api/books/{book_id}", response_model=Book, tags=["Books"])
66 async def update_book(book_id: int, updated_book: Book):
67     for index, book in enumerate(books_db):
68         if book.id == book_id:
69             updated_book.id = book_id
70             books_db[index] = updated_book
71             return updated_book
72     raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"Book with ID {book_id} not found")
73
74 @app.patch("/api/books/{book_id}", response_model=Book, tags=["Books"])
75 async def partial_update_book(book_id: int, book_update: BookUpdate):
76     for book in books_db:
77         if book.id == book_id:
78             update_data = book_update.model_dump(exclude_unset=True)
79             for field, value in update_data.items():
80                 setattr(book, field, value)
81             return book
82     raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"Book with ID {book_id} not found")
83
84 @app.delete("/api/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT, tags=["Books"])
85 async def delete_book(book_id: int):
86     for index, book in enumerate(books_db):
87         if book.id == book_id:
88             books_db.pop(index)
89             return
90     raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"Book with ID {book_id} not found")
91
92 if __name__ == "__main__":
93     import uvicorn
94     uvicorn.run(app, host="0.0.0.0", port=8000)

```

3. Запуск приложения

```
PROBLEMS  TERMINAL  OUTPUT  PORTS  DEBUG CONSOLE  uvicorn ⚠ + v [
(venv) PS C:\Users\Admin\Documents\fastapi_books> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Admin\\Documents\\fastapi_books']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [8740] using WatchFiles
INFO: Started server process [1780]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:39759 - "GET / HTTP/1.1" 200 OK
INFO: 127.0.0.1:38463 - "GET /api/books HTTP/1.1" 200 OK
█
```

<http://127.0.0.1:8000/>

```
127.0.0.1:8000  x +
127.0.0.1:8000
Pretty-print ☐
{"message":"Welcome to Books API!","docs":"/docs","redoc":"/redoc"}
```

<http://127.0.0.1:8000/api/books>

```
127.0.0.1:8000/api/books  x +
127.0.0.1:8000/api/books
Pretty-print ☐
[{"id":1,"title":"War and Peace","author":"Leo Tolstoy","year":1869,"isbn":"9785170987654"},{"id":2,"title":"Crime and Punishment","author":"Fyodor Dostoevsky","year":1866,"isbn":"9785170876543"},{"id":3,"title":"Eugene Onegin","author":"Alexander Pushkin","year":1833,"isbn":"9785170765432"}]
```

<http://127.0.0.1:8000/docs>

Books API

1.0.0

OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET

/ Root

Books

GET

/api/books

Get Books

POST

/api/books

Create Book

GET

/api/books/{book_id}

Get Book

PUT

/api/books/{book_id}

Update Book

PATCH

/api/books/{book_id}

Partial Update Book

DELETE

/api/books/{book_id}

Delete Book

Schemas

Book > Expand all object

BookUpdate > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

<http://127.0.0.1:8000/redoc>

Books API - ReDoc

127.0.0.1:8000/redoc

Search...

Root >

Books >

Books API (1.0.0)

Download OpenAPI specification: [Download](#)

REST API for managing a book library

Root

Root

Responses

> 200 Successful Response

Books

Get Books

Responses

> 200 Successful Response

Create Book

REQUEST BODY SCHEMA: application/json

required

id >

Id (integer) or Id (null) (Id)

title >

string (Title) [1..200] characters

Book title

author >

string (Author) [1..100] characters

Book author

year >

integer (Year) [1000..2025]

Publication year

isbn >

Isbn (string) or Isbn (null) (Isbn)

Book ISBN

Responses

> 201 Successful Response

> 422 Validation Error

GET /

Response samples

200

Content type

application/json

null

Copy

GET /api/books

Response samples

200

Content type

application/json

Copy Expand all Collapse all

[{"author": "Mikhail Bulgakov", "isbn": "9785170123456", "title": "The Master and Margarita", "year": 1967}]

POST /api/books

Request samples

Payload

Content type

application/json

{ "author": "Mikhail Bulgakov", "isbn": "9785170123456", "title": "The Master and Margarita", "year": 1967 }

Response samples

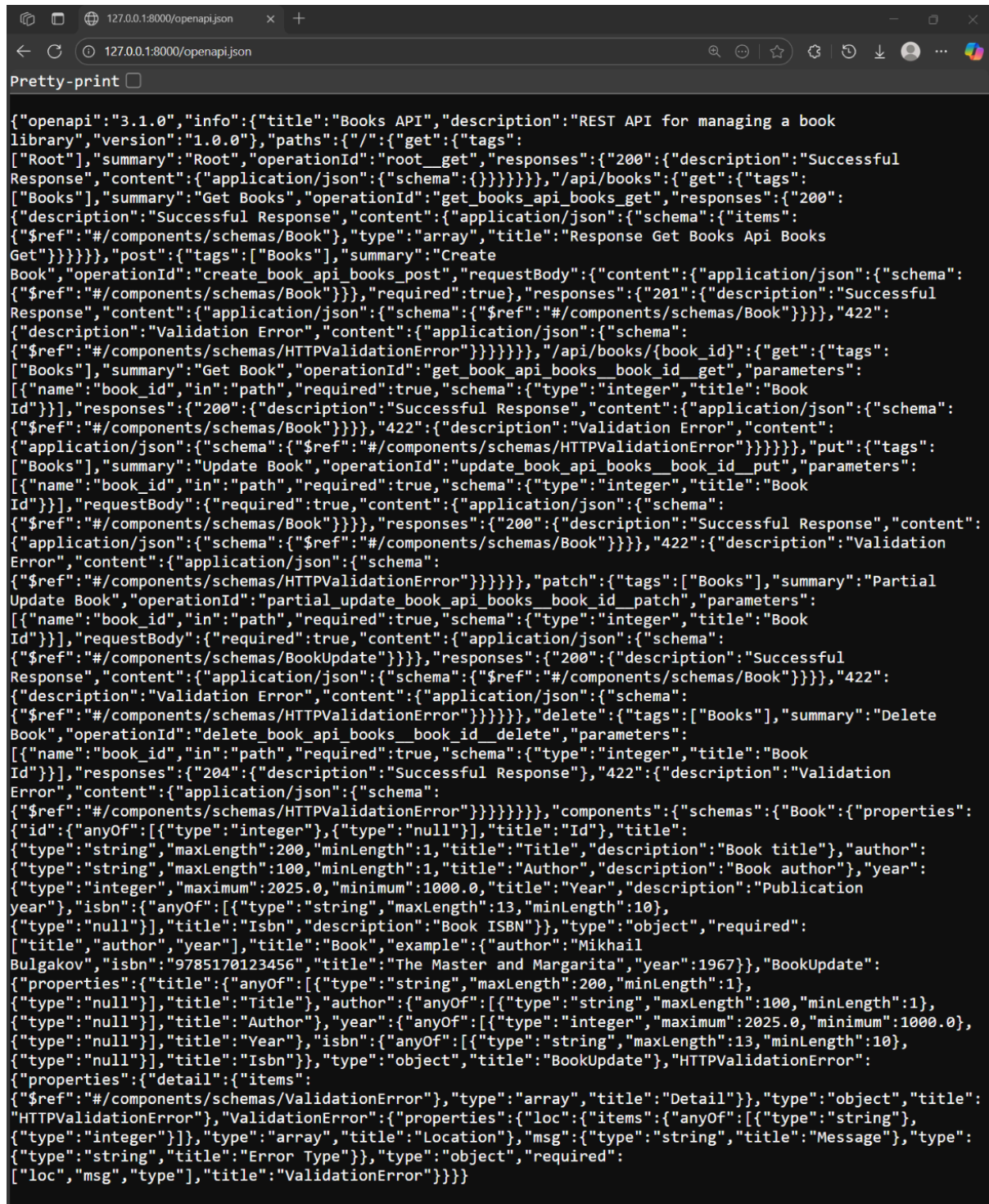
201 422

Content type

application/json

{ "author": "Mikhail Bulgakov", "isbn": "9785170123456", "title": "The Master and Margarita", "year": 1967 }

API docs by Redocly



Задание 2: Тестирование API с помощью Swagger UI

4. Получение списка книг

Books API1.0.0OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET / Root

Books

GET /api/booksGet Books

Parameters

No parameters

ExecuteClear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:8000/api/books' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8000/api/books

Server response

CodeDetails

200

Response body

{\n \"id\": 1,\n \"title\": \"War and Peace\",\n \"author\": \"Leo Tolstoy\",\n \"year\": 1869,\n \"isbn\": \"9785170987654\"\n},\n{\n \"id\": 2,\n \"title\": \"Crime and Punishment\",\n \"author\": \"Fyodor Dostoevsky\",\n \"year\": 1866,\n \"isbn\": \"9785170876543\"\n},\n{\n \"id\": 3,\n \"title\": \"Eugene Onegin\",\n \"author\": \"Alexander Pushkin\",\n \"year\": 1833,\n \"isbn\": \"9785170765432\"\n}\n}

Download

Response headers

content-length: 293\ncontent-type: application/json\ndate: Fri, 24 Oct 2025 18:17:27 GMT\nserver: uvicorn

Responses

CodeDescriptionLinks

200Successful ResponseNo links

Media type

application/json

ControlsAccept header

Example ValueSchema

{\n \"author\": \"Mikhail Bulgakov\",\n \"isbn\": \"9785170123456\",\n \"title\": \"The Master and Margarita\",\n \"year\": 1967\n}\n}

POST /api/booksCreate Book

GET /api/books/{book_id}Get Book

PUT /api/books/{book_id}Update Book

5. Получение книги по ID

Books API - Swagger UI

127.0.0.1:8000/docs#/Books/get_book_api_books_book_id_get

Root

GET / Root

Books

GET /api/books Get Books

POST /api/books Create Book

GET /api/books/{book_id} Get Book

Parameters

Cancel

Name

Description

book_id

required

integer

(path)

1

Execute

Clear

Responses

Curl

curl -X 'GET' \ 'http://127.0.0.1:8000/api/books/1' \ -H 'accept: application/json'

Request URL

http://127.0.0.1:8000/api/books/1

Server response

Code

Details

200

Response body

{

"id": 1,

"title": "War and Peace",

"author": "Leo Tolstoy",

"year": 1869,

"isbn": "9785170987654"

}

Response headers

content-length: 90

content-type: application/json

date: Fri, 24 Oct 2025 18:20:39 GMT

server: uvicorn

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header.

Example Value

Schema

{

"author": "Mikhail Bulgakov",

"isbn": "9785170123456",

"title": "The Master and Margarita",

"year": 1967

}

422

Validation Error

No links

Media type

application/json

Example Value

Schema

{

"detail": [

{

"loc": [

"string",

0

],

"msg": "string",

"type": "string"

}

]

}

PUT /api/books/{book_id} Update Book

6. Создание новой книги

Books API1.0.0OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET / Root

Books

GET /api/books Get Books

POST /api/books Create Book

Parameters

No parameters

Request bodyrequiredapplication/json

Edit Value | Schema

```
{  "author": "Leo Tolstoy",  "isbn": "9785170654321",  "title": "Anna Karenina",  "year": 1877}
```

Execute

Responses

Code	Description	Links
201	Successful Response	No links
	Media type: application/json	
	Controls Accept header	
	Example Value Schema	
	<pre>{ "author": "Mikhail Bulgakov", "isbn": "9785170123456", "title": "The Master and Margarita", "year": 1967}</pre>	
422	Validation Error	No links
	Media type: application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string" }]}</pre>	

GET /api/books/{book_id} Get Book

PUT /api/books/{book_id} Update Book

PATCH /api/books/{book_id} Partial Update Book

Books API1.0.0OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET / Root

Books

GET /api/books Get Books

POST /api/books Create Book

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "author": "Leo Tolstoy",
  "isbn": "9785170654321",
  "title": "Anna Karenina",
  "year": 1877
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/api/books' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "author": "Leo Tolstoy",
  "isbn": "9785170654321",
  "title": "Anna Karenina",
  "year": 1877
}'
```

Request URL

http://127.0.0.1:8000/api/books

Server response

Code

Details

201

Response body

```
{
  "id": 4,
  "title": "Anna Karenina",
  "author": "Leo Tolstoy",
  "year": 1877,
  "isbn": "9785170654321"
}
```

Download

Response headers

```
content-length: 90
content-type: application/json
date: Fri, 24 Oct 2025 18:27:11 GMT
server: uvicorn
```

Responses

Code

Description

Links

201

Successful Response

No links

Media type

application/json

Controls Accept header.

Example Value | Schema

7. Частичное обновление книги

Books API - Swagger UI

127.0.0.1:8000/docs/#/Books/partial_update_book_api_books_book_id_patch

Books

GET /api/books Get Books

POST /api/books Create Book

GET /api/books/{book_id} Get Book

PUT /api/books/{book_id} Update Book

PATCH /api/books/{book_id} Partial Update Book

Parameters

Cancel

Reset

Name

Description

book_id required

integer (path)

1

Request body required

application/json

Edit Value

Schema

{

"year": 1869

}

Execute

Clear

Responses

Curl

curl -X 'PATCH' \

'http://127.0.0.1:8000/api/books/1' \

-H 'accept: application/json' \

-H 'Content-Type: application/json' \

-d '{

"year": 1869

}'

Request URL

http://127.0.0.1:8000/api/books/1

Server response

Code

Details

200

Response body

{

"id": 1,

"title": "War and Peace",

"author": "Leo Tolstoy",

"year": 1869,

"isbn": "9785170987654"

}

Response headers

content-length: 90

content-type: application/json

date: Fri, 24 Oct 2025 18:34:01 GMT

server: uvicorn

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls

Accept header

Example Value

Schema

{

"author": "Mikhail Bulgakov",

"isbn": "9785170123456",

"title": "The Master and Margarita",

}

8. Удаление книги

Books API1.0.0OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET / Root

Books

GET /api/books Get Books

POST /api/books Create Book

GET /api/books/{book_id} Get Book

PUT /api/books/{book_id} Update Book

PATCH /api/books/{book_id} Partial Update Book

DELETE /api/books/{book_id} Delete Book

Parameters

Cancel

Name	Description
book_id * required	
integer	
(path)	

Execute

Clear

Responses

Curl

curl -X 'DELETE' \ 'http://127.0.0.1:8000/api/books/2' \ -H 'accept: */*'

Request URL

http://127.0.0.1:8000/api/books/2

Server response

Code	Details
204	Response headers content-type: application/jsondate: Fri, 24 Oct 2025 18:31:03 GMTserver: uvicorn

Responses

Code	Description	Links
204	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Example Value | Schema

```
{  "detail": [    {      "loc": [        "string",        @      ],      "msg": "string",      "type": "string"    }  ]}
```

Schemas

9. Тестирование валидации

Books API - Swagger UI

127.0.0.1:8000/docs#/Books/create_book_api_books_post

POST /api/books Create Book

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "author": "Test Author",  "isbn": "1234567890",  "title": "Test Book",  "year": 3000}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  http://127.0.0.1:8000/api/books \  -H 'accept: application/json' \  -H 'content-type: application/json' \  -d '{  "author": "Test Author",  "isbn": "1234567890",  "title": "Test Book",  "year": 3000  }'
```

Request URL

http://127.0.0.1:8000/api/books

Server response

Code | Details

422

Error: Unprocessable Entity

Response body

```
{  "detail": [    {      "type": "less_than_equal",      "loc": [        "body",        "year"      ],      "msg": "Input should be less than or equal to 2025",      "input": 3000,      "ctx": {        "le": 2025      }    }  ]}
```

Response headers

```
content-length: 143  content-type: application/json  date: Fri, 24 Oct 2025 18:40:17 GMT  server: uvicorn
```

Responses

Code | Description | Links

201

Successful Response

No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{  "author": "Mikhail Bulgakov",  "isbn": "9785170123456",  "title": "The Master and Margarita",  "year": 1967}
```

422

Validation Error

No links

Media type

application/json

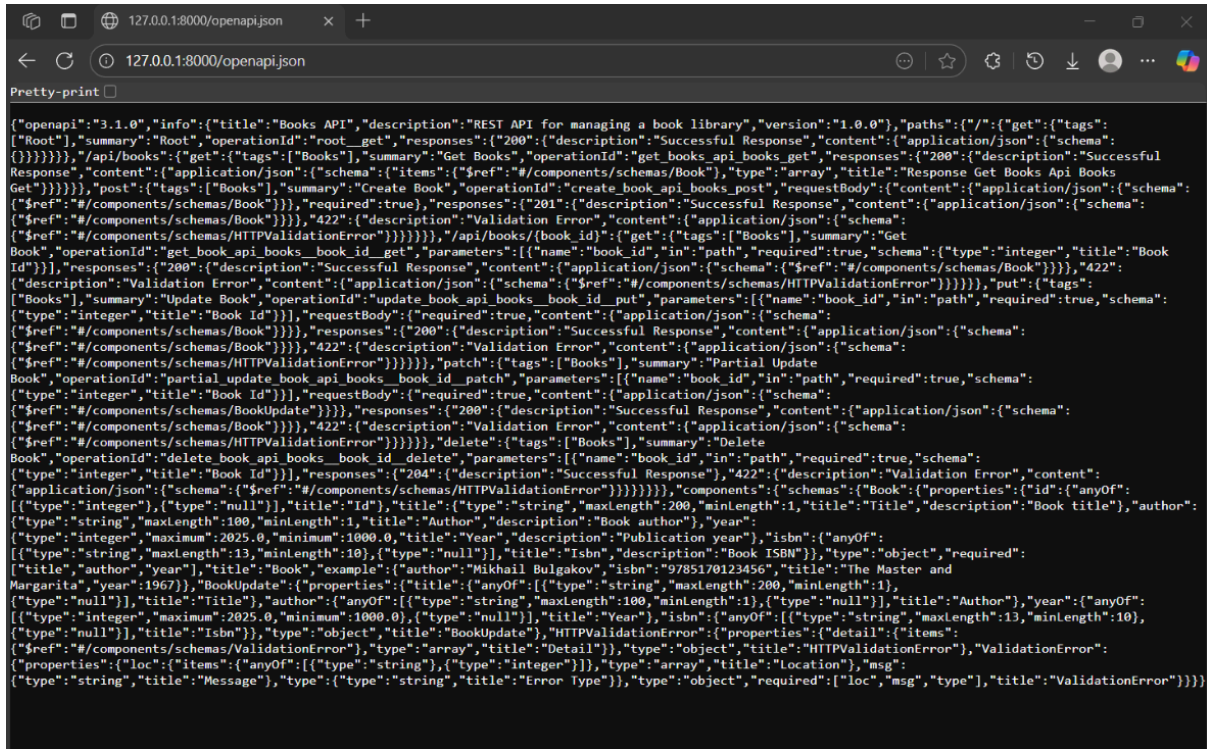
Example Value | Schema

```
{  "detail": [    {      "loc": [        "string",        0      ],      "msg": "string",      "type": "string"    }  ]}
```

GET /api/books/{book_id} Get Book

Задание 3: Изучение OpenAPI спецификации

10. Автоматически сгенерированная OpenAPI спецификация в формате JSON



```
{
  "openapi": "3.1.0",
  "info": {
    "title": "Books API",
    "description": "REST API for managing a book library",
    "version": "1.0.0"
  },
  "paths": {
    "/": {
      "get": {
        "tags": [
          "Root"
        ],
        "summary": "Root",
        "operationId": "root_get",
        "responses": {
          "200": {
            "description": "Successful Response",
            "content": {
              "application/json": {
                "schema": {}
              }
            }
          }
        }
      }
    },
    "/api/books": {
      "get": {
        "tags": [
          "Books"
        ],
        "summary": "Get Books",
        "operationId": "get_books_api_books_get",
        "responses": {
          "200": {
            "description": "Successful Response",
            "content": {
              "application/json": {
                "schema": {
                  "items": {
                    "$ref": "#/components/schemas/Book"
                  },
                  "type": "array"
                }
              }
            },
            "title": "Response Get Books Api Books Get"
          }
        },
        "post": {
          "tags": [
          "Books"
          ],
          "summary": "Create Book",
          "operationId": "create_book_api_books_post",
          "requestBody": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Book"
                }
              }
            },
            "required": true
          },
          "responses": {
            "201": {
              "description": "Successful Response",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/Book"
                  }
                }
              },
              "422": {
                "description": "Validation Error",
                "content": {
                  "application/json": {
                    "schema": {
                      "$ref": "#/components/schemas/HTTPValidationError"
                    }
                  }
                }
              }
            }
          },
          "/api/books/{book_id}": {
            "get": {
              "tags": [
                "Books"
              ],
              "summary": "Get Book",
              "operationId": "get_book_api_books_book_id_get",
              "parameters": [
                {
                  "name": "book_id",
                  "in": "path",
                  "required": true,
                  "schema": {
                    "type": "integer",
                    "title": "Book Id"
                  }
                }
              ],
              "responses": {
                "200": {
                  "description": "Successful Response",
                  "content": {
                    "application/json": {
                      "schema": {
                        "$ref": "#/components/schemas/Book"
                      }
                    }
                  },
                  "422": {
                    "description": "Validation Error",
                    "content": {
                      "application/json": {
                        "schema": {
                          "$ref": "#/components/schemas/HTTPValidationError"
                        }
                      }
                    }
                  }
                }
              },
              "put": {
                "tags": [
                  "Books"
                ],
                "summary": "Update Book",
                "operationId": "update_book_api_books_book_id_put",
                "parameters": [
                  {
                    "name": "book_id",
                    "in": "path",
                    "required": true,
                    "schema": {
                      "type": "integer",
                      "title": "Book Id"
                    }
                  }
                ],
                "requestBody": {
                  "required": true,
                  "content": {
                    "application/json": {
                      "schema": {
                        "$ref": "#/components/schemas/BookUpdate"
                      }
                    }
                  }
                },
                "responses": {
                  "200": {
                    "description": "Successful Response",
                    "content": {
                      "application/json": {
                        "schema": {
                          "$ref": "#/components/schemas/Book"
                        }
                      }
                    },
                    "422": {
                      "description": "Validation Error",
                      "content": {
                        "application/json": {
                          "schema": {
                            "$ref": "#/components/schemas/HTTPValidationError"
                          }
                        }
                      }
                    }
                }
              },
              "patch": {
                "tags": [
                  "Books"
                ],
                "summary": "Partial Update Book",
                "operationId": "partial_update_book_api_books_book_id_patch",
                "parameters": [
                  {
                    "name": "book_id",
                    "in": "path",
                    "required": true,
                    "schema": {
                      "type": "integer",
                      "title": "Book Id"
                    }
                  }
                ],
                "requestBody": {
                  "required": true,
                  "content": {
                    "application/json": {
                      "schema": {
                        "$ref": "#/components/schemas/BookUpdate"
                      }
                    }
                  }
                },
                "responses": {
                  "200": {
                    "description": "Successful Response",
                    "content": {
                      "application/json": {
                        "schema": {
                          "$ref": "#/components/schemas/Book"
                        }
                      }
                    },
                    "422": {
                      "description": "Validation Error",
                      "content": {
                        "application/json": {
                          "schema": {
                            "$ref": "#/components/schemas/HTTPValidationError"
                          }
                        }
                      }
                    }
                }
              },
              "delete": {
                "tags": [
                  "Books"
                ],
                "summary": "Delete Book",
                "operationId": "delete_book_api_books_book_id_delete",
                "parameters": [
                  {
                    "name": "book_id",
                    "in": "path",
                    "required": true,
                    "schema": {
                      "type": "integer",
                      "title": "Book Id"
                    }
                  }
                ],
                "responses": {
                  "204": {
                    "description": "Successful Response",
                    "content": {
                      "application/json": {
                        "schema": {
                          "$ref": "#/components/schemas/HTTPValidationError"
                        }
                      }
                    },
                    "422": {
                      "description": "Validation Error",
                      "content": {
                        "application/json": {
                          "schema": {
                            "$ref": "#/components/schemas/HTTPValidationError"
                          }
                        }
                      }
                    }
                }
              }
            }
          }
        }
      }
    }
  },
  "components": {
    "schemas": {
      "Book": {
        "properties": {
          "id": {
            "anyOf": [
              {
                "type": "integer",
                "title": "Id"
              },
              {
                "type": "string",
                "maxLength": 200,
                "minLength": 1,
                "title": "Title",
                "description": "Book title",
                "author": {
                  "type": "string",
                  "maxLength": 100,
                  "minLength": 1,
                  "title": "Author",
                  "description": "Book author",
                  "year": {
                    "type": "integer",
                    "maximum": 2025.0,
                    "minimum": 1000.0,
                    "title": "Year",
                    "description": "Publication year",
                    "isbn": {
                      "anyOf": [
                        {
                          "type": "string",
                          "maxLength": 13,
                          "minLength": 10,
                          "type": "null"
                        },
                        {
                          "type": "string",
                          "maxLength": 13,
                          "minLength": 10,
                          "type": "object",
                          "required": [
                            "title",
                            "author",
                            "year"
                          ],
                          "title": "Book",
                          "example": {
                            "author": "Mikhail Bulgakov",
                            "isbn": "9785170123456",
                            "title": "The Master and Margarita",
                            "year": 1967
                          },
                          "BookUpdate": {
                            "properties": {
                              "title": {
                                "anyOf": [
                                  {
                                    "type": "string",
                                    "maxLength": 200,
                                    "minLength": 1
                                  },
                                  {
                                    "type": "null"
                                  }
                                ],
                                "title": "Title",
                                "author": {
                                  "anyOf": [
                                    {
                                      "type": "string",
                                      "maxLength": 100,
                                      "minLength": 1,
                                      "type": "null"
                                    },
                                    {
                                      "type": "integer",
                                      "maximum": 2025.0,
                                      "minimum": 1000.0,
                                      "type": "null"
                                    }
                                  ],
                                      "title": "Author",
                                      "year": {
                                        "anyOf": [
                                          {
                                            "type": "string",
                                            "maxLength": 13,
                                            "minLength": 10,
                                            "type": "null"
                                          },
                                          {
                                            "type": "string",
                                            "maxLength": 13,
                                            "minLength": 10,
                                            "type": "object",
                                            "title": "BookUpdate",
                                            "HTTPValidationError": {
                                              "properties": {
                                                "detail": {
                                                  "items": {
                                                    "anyOf": [
                                                      {
                                                        "type": "string"
                                                      },
                                                      {
                                                        "type": "integer"
                                                      }
                                                    ],
                                                    "type": "array",
                                                    "title": "Location",
                                                    "msg": {
                                                      "type": "string",
                                                      "title": "Message",
                                                      "type": {
                                                        "type": "string",
                                                        "title": "Error type"
                                                      },
                                                      "type": "object",
                                                      "required": [
                                                        "loc",
                                                        "msg",
                                                        "type"
                                                      ],
                                                      "title": "ValidationError"
                                                    }
                                                  }
                                                }
                                              }
                                            }
                                          }
                                        ]
                                      }
                                    }
                                  ]
                                }
                              }
                            }
                          }
                        }
                      }
                    }
                  }
                }
              }
            ]
          }
        }
      }
    }
  }
}
```

Задание 4: Расширение API (самостоятельная работа)

11. Добавление фильтрации и поиска

```
from typing import Optional
```

```
47 @app.get("/api/books", response_model=List[Book], tags=["Books"])
48 async def get_books(
49     author: Optional[str] = None,
50     year_from: Optional[int] = None,
51     year_to: Optional[int] = None
52 ):
53     filtered_books = books_db
54
55     if author:
56         filtered_books = [b for b in filtered_books if author.lower() in b.author.lower()]
57     if year_from:
58         filtered_books = [b for b in filtered_books if b.year >= year_from]
59     if year_to:
60         filtered_books = [b for b in filtered_books if b.year <= year_to]
61
62     return filtered_books
```

Тестирование:

Поиск по автору:

Books API1.0.0OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET / Root

Books

GET /api/books Get Books

Parameters

Name

Description

author

string | (string | null) (query)

Tolstoy

year_from

integer | (integer | null) (query)

year_from

year_to

integer | (integer | null) (query)

year_to

Execute

Clear

Responses

Curl

curl -X 'GET' \nhttp://127.0.0.1:8000/api/books?author=Tolstoy' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8000/api/books?author=Tolstoy

Server response

Code

Details

200

Response body

{\n \"id\": 1,\n \"title\": \"War and Peace\",\n \"author\": \"Leo Tolstoy\",\n \"year\": 1869,\n \"isbn\": \"9785170867654\"\n}

Download

Response headers

content-length: 92\ncontent-type: application/json\ndate: Fri, 24 Oct 2025 19:52:26 GMT\nserver: uvicorn

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header.

Example Value

Schema

{\n \"author\": \"Mikhail Bulgakov\",\n \"isbn\": \"9785170123456\",\n \"title\": \"The Master and Margarita\",\n \"year\": 1967\n}

422

Validation Error

No links

Фильтрация по годам:

Books API1.0.0OAS 3.1

/openapi.json

REST API for managing a book library

Root

GET / Root

Books

GET /api/booksGet Books

Parameters

Cancel

Name	Description
author	author
year_from	1860
year_to	1870

ExecuteClear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:8000/api/books?year_from=1860&year_to=1870' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8000/api/books?year_from=1860&year_to=1870

Server response

Code

Details

200

Response body

{\n {\n \"id\": 1,\n \"title\": \"War and Peace\",\n \"author\": \"Leo Tolstoy\",\n \"year\": 1869,\n \"isbn\": \"9785170987654\"\n },\n {\n \"id\": 2,\n \"title\": \"Crime and Punishment\",\n \"author\": \"Fyodor Dostoevsky\",\n \"year\": 1866,\n \"isbn\": \"9785170876543\"\n }\n}

Download

Response headers

content-length: 196\ncontent-type: application/json\ndate: Fri, 24 Oct 2025 19:53:34 GMT\nserver: unicorn

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

{\n \"author\": \"Mikhail Bulgakov\",\n \"isbn\": \"9785170123456\",\n \"year\": 1928\n}

12. Добавление пагинации

```
47 @app.get("/api/books", response_model=List[Book], tags=["Books"])
48 async def get_books(
49     # author: Optional[str] = None,
50     # year_from: Optional[int] = None,
51     # year_to: Optional[int] = None
52     skip: int = 0,
53     limit: int = 10,
54     author: Optional[str] = None
55 ):
56     filtered_books = books_db
57
58     if author:
59         filtered_books = [b for b in filtered_books if author.lower() in b.author.lower()]
60     # if year_from:
61     #     filtered_books = [b for b in filtered_books if b.year >= year_from]
62     # if year_to:
63     #     filtered_books = [b for b in filtered_books if b.year <= year_to]
64
65     return filtered_books[skip: skip + limit]
```

13. Добавление статистики

```
from collections import Counter
```

```
110 @app.get("/api/books/stats", tags=["Statistics"])
111 async def get_statistics():
112     total_books = len(books_db)
113     authors = Counter(book.author for book in books_db)
114     centuries = Counter(book.year // 100 + 1 for book in books_db)
115
116     return {
117         "total_books": total_books,
118         "books_by_author": dict(authors),
119         "books_by_century": {f"{century} century": count for century, count in centuries.items()}
120     }
```

Задание 5: Добавление базы данных SQLite

14. Установка SQLAlchemy

```
PROBLEMS  TERMINAL  OUTPUT  PORTS  DEBUG CONSOLE
(venv) PS C:\Users\Admin\Documents\fastapi_books> pip install sqlalchemy
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.44-cp312-cp312-win_amd64.whl.metadata (9.8 kB)
Collecting greenlet>=1 (from sqlalchemy)
  Downloading greenlet-3.2.4-cp312-cp312-win_amd64.whl.metadata (4.2 kB)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\admin\documents\fastapi_books\venv\lib\site-packages (from sqlalchemy) (4.15.0)
Downloading sqlalchemy-2.0.44-cp312-cp312-win_amd64.whl (2.1 MB)
  2.1/2.1 MB 1.1 MB/s eta 0:00:00
Downloading greenlet-3.2.4-cp312-cp312-win_amd64.whl (299 kB)
Installing collected packages: greenlet, sqlalchemy
Successfully installed greenlet-3.2.4 sqlalchemy-2.0.44
```

15. Создание database.py

```

database.py X
database.py > get_db
1  from sqlalchemy import create_engine, Column, Integer, String
2  from sqlalchemy.ext.declarative import declarative_base
3  from sqlalchemy.orm import sessionmaker
4
5  SQLALCHEMY_DATABASE_URL = "sqlite:///./books.db"
6  engine = create_engine(SQLALCHEMY_DATABASE_URL, connect_args={"check_same_thread": False})
7
8  SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
9
10 Base = declarative_base()
11
12 class BookDB(Base):
13     __tablename__ = "books"
14
15     id = Column(Integer, primary_key=True, index=True)
16     title = Column(String(200), nullable=False)
17     author = Column(String(100), nullable=False)
18     year = Column(Integer, nullable=False)
19     isbn = Column(String(13), nullable=True)
20
21     Base.metadata.create_all(bind=engine)
22
23 def get_db():
24     db = SessionLocal()
25     try:
26         yield db
27     finally:
28         db.close()

```

16. Модификация main.py

```

7  from fastapi import Depends
8  from sqlalchemy.orm import Session
9  from database import get_db, BookDB

```

```

51 @app.get("/api/books", response_model=List[Book], tags=["Books"])
52 v async def get_books(db: Session = Depends(get_db)):
53     books = db.query(BookDB).all()
54     return books

```

```

81 @app.post("/api/books", response_model=Book, status_code=status.HTTP_201_CREATED, tags=["Books"])
82 async def create_book(book: Book, db: Session = Depends(get_db)):
83     db_book = BookDB(**book.model_dump(exclude={"id"}))
84     db.add(db_book)
85     db.commit()
86     db.refresh(db_book)
87     return db_book

```

Задание 6: Добавление аутентификации

17. Создание файла auth.py

```
auth.py ×  
auth.py > verify_api_key  
1  from fastapi import Security, HTTPException, status  
2  from fastapi.security import APIKeyHeader  
3  
4  API_KEY = "secret-api-key-12345"  
5  api_key_header = APIKeyHeader(name="X-API-Key")  
6  
7  async def verify_api_key(api_key: str = Security(api_key_header)):  
8      if api_key != API_KEY:  
9          raise HTTPException(  
10             status_code=status.HTTP_403_FORBIDDEN,  
11             detail="Invalid API key"  
12         )  
13     return api_key
```

18. Защита эндпоинтов

```
from auth import verify_api_key
```

```
82  @app.post("/api/books", response_model=Book, status_code=status.HTTP_201_CREATED, tags=["Books"])  
83  async def create_book(book: Book, db: Session = Depends(get_db), api_key: str = Depends(verify_api_key)):  
84      db_book = BookDB(**book.model_dump(exclude={"id"}))  
85      db.add(db_book)  
86      db.commit()  
87      db.refresh(db_book)  
88      return db_book
```